

Echanges avec IA générative Gemini

1.0 Question DAO extrait code python, exemple fichier d'environnement

Pouvez-vous me donner un extrait de code python dao, juste la connexion dans un try except qui me permette de me connecter à un base mariadb.

J'ai besoin aussi d'un exemple de fichier d'environnement .env à la base de mon projet.

Est ce que j'aurai aussi besoin de librairie particulière python pour lire ce fichier et lesquelles ?

1.1 Réponse de Gemini DAO extrait code python, exemple fichier d'environnement

Oui, vous aurez besoin de deux bibliothèques Python principales :

- mariadb (ou mysql-connector-python) : le pilote pour interagir avec la base de données MariaDB.
- python-dotenv : pour charger automatiquement les variables du fichier .env dans votre environnement Python.

Installez-les via votre terminal :

```
pip install mariadb python-dotenv
```

Voici le fichier .env à placer à la racine de votre projet :

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=mon_utilisateur
DB_PASSWORD=mon_mot_de_passe
DB_NAME=ma_base_de_donnees
```

Voici un extrait de code Python pour initialiser la connexion dans un bloc try...except :

```
import os
import mariadb
from dotenv import load_dotenv

# Chargement des variables d'environnement depuis le fichier .env
load_dotenv()

def get_db_connection():
    connection = None
    try:
        connection = mariadb.connect(
            host=os.getenv("DB_HOST"),
            port=int(os.getenv("DB_PORT", 3306)),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD"),
            database=os.getenv("DB_NAME")
        )
        print("Connexion à MariaDB réussie !")
        return connection
    except mariadb.Error as e:
        print(f"Erreur lors de la connexion à MariaDB : {e}")
        return None

# Exemple d'utilisation
if __name__ == "__main__":
    conn = get_db_connection()
    if conn:
        # Pensez à toujours fermer la connexion une fois terminé
        conn.close()
```

2.0 Question erreurs mariaDB

Quelles sont les erreurs courantes mariadb leurs chiffres et ce que cela signifie ?

2.1 Réponse Gemini erreurs mariaDB

Voici les erreurs courantes rencontrées avec MariaDB, classées par domaine :

2.1.1 Erreurs d'authentification et de connexion

Code	Libellé MariaDB	Signification / Cause courante
1045	ER_ACCESS_DENIED_ERROR	Identifiants incorrects : Nom d'utilisateur, mot de passe ou hôte d'origine non autorisé.
1044	ER_DBACCESS_DENIED_ERROR	Droits insuffisants : L'utilisateur est authentifié mais n'a pas accès à la base de données visée.
1040	ER_CON_COUNT_ERROR	Trop de connexions : Le nombre maximal de connexions simultanées (max_connections) est atteint.
2002	CR_CONNECTION_ERROR	Serveur inaccessible : MariaDB ne tourne pas ou le chemin du socket Unix / port est incorrect.
2003	CR_CONN_HOST_ERROR	Connexion refusée : Problème de pare-feu, port 3306 fermé ou serveur configuré pour écouter sur 127.0.0.1 uniquement.

2.1.2 Erreurs de syntaxe et de structure SQL

Code	Libellé MariaDB	Signification / Cause courante
1064	ER_PARSE_ERROR	Erreur de syntaxe : Mot-clé SQL mal orthographié, virgule en trop ou guillemet non fermé.
1046	ER_NO_DB_ERROR	Aucune base sélectionnée : Omission de la commande USE nom_base avant d'exécuter la requête.
1146	ER_NO_SUCH_TABLE	Table inexistante : Faute dans le nom de la table ou base de données incorrecte.
1054	ER_BAD_FIELD_ERROR	Colonne inconnue : La colonne spécifiée n'existe pas dans la table ciblée.

2.1.3. Erreurs de contraintes et d'exécution

Code	Libellé MariaDB	Signification / Cause courante
1062	ER_DUP_ENTRY	Doublon : Tentative d'insertion d'une valeur existante dans une colonne avec une contrainte UNIQUE ou PRIMARY KEY.
1451 / 1452	ER_ROW_IS_REFERENCED..	Violation de clé étrangère : Tentative d'insertion avec une référence inexistante (1452) ou de suppression d'un parent lié (1451).
2006	CR_SERVER_GONE_AWAY	Déconnexion pendant la requête : Le serveur a coupé le lien (dépassement de max_allowed_packet ou wait_timeout).
1114	ER_RECORD_FILE_FULL	Disque ou table plein(e) : Espace disque insuffisant sur le serveur.

Question 3.0 intégration continue avec pre-commit

Je souhaite utiliser le fichier requirements.txt suivant pour installer un environnement virtuel python et faire de l'intégration continue. J'aurai besoin d'un petit tutoriel pour utiliser le framework pre-commit, qu'il se déclenche avant un commit et le niveau de sévérité soit bas pour radon et bandit et qu'ils ne se déclenchent pas des faux positifs. Voici mon fichier :

```
# --- Chiffrement & Sécurité ---
# Dépendances bas niveau pour cryptography
cffi~=2.0
pycparser~=2.23
# SSL et chiffrement
cryptography~=46.0

# --- Base de données ---
PyMySQL~=1.1
types-PyMySQL~=1.1

# --- Configuration & Environnement ---
python-dotenv~=1.0

# --- Documentation ---
pydocstyle~=6.3
sphinx~=7.2
mkdocs~=1.5
mkdocstrings[python]~=0.24

# --- Qualité de code & Sécurité ---
mypy~=1.8.0
flake8~=7.0
bandit~=1.7
radon~=6.0

# --- Intégration continue ---
pre-commit~=4.1.0
```

3.1. Réponse Gemini intégration continue avec pre-commit

Voici ton fichier `requirements.txt` nettoyé (les doublons `python-dotenv` et `mypy` ont été retirés) avec l'ajout de `pre-commit~=4.1.0` (la version stable actuelle).

Pour régler la sévérité de **Bandit** et **Radon** afin d'éviter les faux positifs, la configuration se fait directement au niveau des arguments passés dans le fichier `.pre-commit-config.yaml`.

3.1.1. Configuration `.pre-commit-config.yaml` (Sévérité modérée)

Pour éviter les alertes trop strictes :

- **Bandit** : On utilise `-ll` (niveau de sévérité **Moyen ou Élevé** uniquement) et `-ii` (niveau de confiance **Moyen ou Élevé** uniquement). On exclut aussi les dossiers de tests.
- **Radon** : On fixe le seuil de complexité à `-n C` (seules les fonctions très complexes de note C, D, E ou F lèveront une alerte, en ignorant les fonctions simples A et B).

Crée le fichier `.pre-commit-config.yaml` à la racine de ton projet :

```
repos:
- repo: local
  hooks:
    - id: flake8
      name: flake8
      entry: flake8
      language: system
      types: [python]
      args: ["--max-line-length=88", "--ignore=E203,W503"]

    - id: mypy
      name: mypy
      entry: mypy
      language: system
      types: [python]
      args: ["--ignore-missing-imports"]

    - id: pydocstyle
      name: pydocstyle
      entry: pydocstyle
      language: system
      types: [python]
      args: ["--match-dir=^(?!tests|docs|\\.venv).*"]

    - id: bandit
      name: bandit (sécurité - sévérité moyenne+)
      entry: bandit
      language: system
      types: [python]
      # -ll : Medium/High severity, -ii : Medium/High confidence
      args: ["-r", "src/", "-ll", "-ii", "-x", "tests/"]

    - id: radon
      name: radon (complexité cyclomatique >= C)
      entry: radon cc
      language: system
      types: [python]
      # -n C : alerte uniquement à partir de la note C (complexité modérée/élevée)
      args: ["src/", "-n", "C", "-a"]
```

3.1.2 Activer pre-commit

Une fois les paquets installés (`pip install -r requirements.txt`), lance :

```
pre-commit install
```

Désormais, `pre-commit` filtrera le code sans bloquer le développement pour des alertes mineures.

4.0 Questino `pyproject.toml`

1. Pourquoi est bien d'avoir un fichier "`pyproject.toml`" dans un projet python ?
2. Qu'est-ce que cela m'apporte ?
3. Est-ce que cela est toujours avantageux alors que mon projet n'est pas un projet nouveau ?
4. Qu'est-ce que cela apporte au niveau de la gestion des outils ?
5. Pouvez-vous me montrer un exemple de `pyproject.toml` ?

4.1 Réponse Gemini :

4.1.1. Pourquoi est-il bien d'avoir un fichier `pyproject.toml` dans un projet Python ?

Avoir un fichier `pyproject.toml` est une bonne pratique car il s'agit du **standard officiel de la communauté Python** (défini par la [PEP 518](#) et la [PEP 621](#)).

Avant sa création, la configuration d'un projet Python était éparpillée entre plusieurs fichiers non standardisés (`setup.py`, `setup.cfg`, `requirements.txt`, `Pipfile`, `.flake8`, `mypy.ini`, etc.). Le fichier `pyproject.toml` résout ce problème en offrant un **point d'entrée unique, déclaratif et lisible** pour tout ce qui concerne le projet.

4.1.2. Qu'est-ce que cela m'apporte ?

1. **Centralisation** : Tu lis et modifies toute la configuration de ton projet au même endroit.
2. **Lisibilité (format TOML)** : Contrairement aux scripts exécutables (`setup.py`) ou aux syntaxes hétérogènes, le TOML est un format de données clair, facile à relire et à éditer sans risquer d'exécuter du code arbitraire.
3. **Compatibilité moderne** : Tous les outils récents de l'écosystème Python (`uv`, `poetry`, `hatch`, `ruff`, `mypy`, `pytest`) le comprennent nativement.
4. **Simplification du packaging** : Si un jour tu souhaites publier ton projet sous forme de paquet (sur PyPI ou en interne), toutes les métadonnées (nom, version, auteurs, dépendances) sont déjà prêtes.

4.1.3. Est-ce que cela est toujours avantageux alors que mon projet n'est pas un projet nouveau ?

Oui, absolument. L'adopter sur un projet existant apporte des bénéfices immédiats :

- **Nettoyage de la racine** : Tu peux supprimer une multitude de fichiers de configuration individuels (`pytest.ini`, `mypy.ini`, `.pydocstyle`, etc.) pour tout regrouper dans un seul fichier.
- **Transition progressive** : Tu n'es pas obligé de tout migrer du jour au lendemain. Tu peux commencer par y mettre la configuration de tes outils de qualité de code (`pytest`, `mypy`, `ruff`), puis basculer la gestion de tes dépendances plus tard.
- **Onboarding facilité** : Un nouveau développeur arrivant sur le projet n'a qu'un seul fichier à consulter pour comprendre la structure, les dépendances et les règles de style du code.

4.1.4. Qu'est-ce que cela apporte au niveau de la gestion des outils ?

- **Isolation et reproductibilité** : Grâce à la section `[build-system]`, Python sait exactement quels outils sont nécessaires pour installer ou construire ton projet sans dépendre de l'environnement global de la machine.
- **Configuration unifiée** (`[tool.nom_du_outil]`) : Chaque outil de ton workflow (linter, formateur, type-checker, moteur de tests) dispose de son propre espace dédié dans le fichier.
- **Partage d'équipe** : L'ensemble de ton équipe de développement partage strictement la même configuration pour les outils de qualité, garantissant un code homogène lors des révisions (`code review`).

4.1.5. Exemple concret de `pyproject.toml`

Voici un exemple complet regroupant métadonnées, dépendances et configurations d'outils :

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

# --- Métadonnées du projet (PEP 621) ---
[project]
name = "mon-projet-python"
version = "0.2.0"
description = "Projet Python avec configuration centralisée"
readme = "README.md"
requires-python = ">=3.10"
authors = [
    { name = "Ton Name", email = "ton.email@example.com" }
]
dependencies = [
    "requests>=2.31.0",
    "pydantic>=2.0.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
    "mypy>=1.8.0",
    "ruff>=0.4.0",
]

# --- Configuration de Pytest ---
[tool.pytest.ini_options]
minversion = "8.0"
addopts = "-ra -q"
testpaths = ["tests"]

# --- Configuration de Mypy (Type Checking) ---
[tool.mypy]
python_version = "3.10"
warn_return_any = true
warn_unused_configs = true
disallow_untyped_defs = true

# --- Configuration de Ruff (Linter & Formatter moderne) ---
[tool.ruff]
line-length = 88
target-version = "py310"

[tool.ruff.lint]
select = [
    "E", "F",      # Erreurs et avertissements Flake8
    "I",           # Tri des imports (isort)
    "D",           # Vérification des docstrings (pydocstyle)
]
ignore = ["D100"] # Exemple : ignorer la vérification de docstring sur les fichiers de
niveau module
```